



Kantonsschule Zürich Nord

Lang- und Kurzgymnasium

Fachmittelschule

Schreiben einer Maturarbeit

Mit dem Textsatzsystem Typst

Eine Anleitung für Einsteiger



Verfasst von

Christian Prim und Lukas Zuberbühler

Version 08.09.2026

Abstract

Dieses Dokument beschreibt das Typst-Template der Kantonsschule Zürich Nord (KZN) für Maturitätsarbeiten und andere schriftliche Arbeiten. Es richtet sich an Schülerinnen und Schüler, die ihre Arbeit mit dem modernen Textsatzsystem Typst verfassen möchten, sowie an Lehrpersonen, die das Template für eigene Dokumente einsetzen.

Im ersten Teil werden die Grundkonzepte von Typst erläutert: Textformatierung mit Markdown, die zentralen Funktionen, sowie das Einbinden von Tabellen, Abbildungen und Formeln.

Der zweite Teil dokumentiert das KZN-Template konkret: Projekterstellung, Anpassung von Layout und Titelseite, Verwaltung der Frontmatter-Blöcke, Literaturreferenzen im BibTeX-Format, Mehrsprachigkeit sowie die Anonymisierungsfunktion für Plagiatsprüfungen.

Im Anhang finden sich vollständige Beispiele für den mathematischen Formelsatz, den Umgang mit diakritischen Zeichen und nicht-lateinischen Schriften, sowie Vorlagen für Einzel- und Mehrfachabbildungen und -tabellen.

Erklärung zur Nutzung von KI

KI-Tools wurden bei der Erstellung dieser Arbeit in folgenden Bereichen eingesetzt:

- Übersetzungen und Sprachbeispiele (Claude Sonnet 4.6)
- Grammatik- und Stilprüfung (Claude Sonnet 4.6)
- Code-Generierung für Beispiele (Claude Sonnet 4.6)

Alle KI-generierten Inhalte wurden sorgfältig überprüft, verifiziert und bei Bedarf angepasst.

Danksagung

Wir danken dem Typst-Entwicklerteam für die Bereitstellung eines modernen, leistungsfähigen und frei zugänglichen Textsatzsystems.

Vorwort

Wer eine Maturarbeit schreibt, soll sich auf den Inhalt konzentrieren können – nicht auf Seitenränder, Schriftgrössen und Titelseiten.

Mit Typst steht ein modernes Textsatzsystem zur Verfügung, das eine klare Trennung von Inhalt und Layout erlaubt. Die Formatierung geschieht automatisch, ist aber dennoch überall anpassbar. Literaturreferenzen können mit Online-Datenbanken wie Mendeley oder Zotero verwaltet und mit minimalem Aufwand in die Arbeit integriert werden. Der Formelsatz für Mathematik, Physik und Chemie ist intuitiv, ebenso Codeblöcke für Informatikarbeiten.

Dieses Template soll Zeit und Nerven sparen, damit beides für das wirklich Wichtige zur Verfügung steht: das Denken, Recherchieren und Schreiben.

Quickstart

Um die eigene Arbeit zu starten, sind mindestens diese Schritte nötig:

- Im Dokument `main.typ` nach dem Kommentar «Allgemeine Angaben» die Informationen wie Titel, Namen etc. anpassen.
- Ein eigenes Bild für die Titelseite in den Ordner `img` hochladen und in der `kznTitlePage()`-Funktion im Dokument `main.typ` anpassen:

```
// Titelseite mit KZN-Gestaltung und Hintergrundbild
#let kznTitlePage() = kzn-titlepage(
...
nord-image: image("img/neuesBild.jpeg", height: 100%),
// Quellenangabe zum Hintergrundbild (beliebiger Textblock)
nord-image-source: [#localize(cover-image) #link("https://www.meineBildquelle.
ch")],
...
)
```

- Die Vorspann-Blöcke im Dokument `main.typ` anpassen und nicht gewünschte löschen. So würde z.B. nur der Abstract-Block gesetzt:

```
#let frontmatter-def = (
  content: (
    myAbstract(),
  ),
  // numbering: "i", // Römische Seitenzahlen im Vorspann
  // footer: kzn-footer(footer-text: []), // Eigene Fusszeile im Vorspann
)
```

- Inhalt in `main-matter.typ` und `appendix.typ` löschen bis auf folgende Zeilen:

```
#import "@preview/kzn-ma:0.1.0": * // Diese Zeile ist immer nötig
#import "@preview/unify:0.7.1": unit, qty, num // Diese Zeile nötig, wenn Formeln
gesetzt werden
#import "@preview/codly:1.3.0": codly, codly-init
#import "@preview/codly-languages:0.1.10": * // Diese beiden Zeilen sind nötig,
wenn Codeblöcke gesetzt werden
```

- Arbeit in `main-matter.typ` und `appendix.typ` schreiben.

Inhaltsverzeichnis

Inhaltsverzeichnis	vii
Abbildungsverzeichnis	ix
Tabellenverzeichnis	x
1 Einleitung	1
1.1 Was ist ein Textsatzsystem wie Typst?	1
1.2 Für welche Arbeiten ist Typst geeignet, für welche eher nicht?	1
1.3 Inhalt dieses Dokuments	1
2 Das Textsatzsystem Typst	2
2.1 Textformatierung	2
2.2 Funktionen für die Formatierung	2
2.2.1 Funktionsaufruf aus dem Text	2
2.2.2 Spezielle Funktionen	3
2.3 Verweise und Fussnoten	4
2.4 Tabellen	4
2.5 Illustrationen / Bilder	5
2.6 Formeln	5
2.7 Einheiten	6
3 Das kzn-ma.typ konkret	7
3.1 Ein neues Projekt anlegen	7
3.2 Anpassen von Textvariablen	7
3.3 Anpassen des Layouts	9
3.3.1 Grundsätzliche Layoutanpassungen	9
3.3.2 Titelseite	10
3.3.3 Definition der Blöcke vor der eigentlichen Arbeit	11
3.3.4 Anhänge	13
3.4 Literaturreferenzen	14
3.5 Mehrsprachigkeit und <code>localize()</code>	16
3.6 Anonymisierung mit <code>copystop</code>	17
3.7 Erweiterungen der <code>#figure</code> -Umgebung	18
4 Diskussion und Ausblick	20
4.1 Weiterentwicklung des Templates	20
4.2 Ressourcen für den Einstieg	20
4.2.1 Offizielle Dokumentation	20
4.2.2 Pakete und Erweiterungen	20
4.2.3 Community und Hilfe	20
4.2.4 Vergleich mit $\text{\LaTeX}$	21
4.2.5 Tutorials und Lernmaterial	21
Literatur	22

A	Formelsatz	23
AA	Grundlegende Mathematik	23
AB	Physikalische Formeln	24
AC	Einheiten mit <code>unify</code>	24
B	Chemische Formeln und Reaktionsgleichungen	26
BA	Summenformeln und Ionen	26
BB	Reaktionsgleichungen	26
BC	Reaktionsgleichung als freistehender Block	27
C	Diakritische Zeichen und nicht-lateinische Schriften	28
CA	Romanische Sprachen	28
CB	Latein	28
CC	Griechisch	29
CD	Arabisch	29
D	Abbildungsbeispiele	31
DA	Einfache Abbildung	31
DB	Abbildung als Grid (mehrere Teilabbildungen)	31
E	Tabellenbeispiele	33
EA	Einfache Tabelle	33
EB	Mehrere Tabellen nebeneinander (Grid)	33

Abbildungsverzeichnis

Abb. 1	Abbildung mit mehreren Teilabbildungen.....	19
---------------	---	----

Tabellenverzeichnis

Tab. 1	Abmessungen einzelner Möbelstücke	5
Tab. 2	Schallgeschwindigkeit unter verschiedenen Bedingungen.	33
Tab. 3	Physikalische Eigenschaften der Planeten des Sonnensystems.	35

1 Einleitung

In diesem Dokument wird die Vorlage (Template) der Kantonsschule Zürich Nord [1] für Maturitätsarbeiten oder andere schriftliche Arbeiten mit dem Textsatzsystem «Typst» [2] vorgestellt. Um einen realistischen Eindruck zu vermitteln, haben wir versucht, das vorliegende Dokument möglichst so wie eine wissenschaftliche Arbeit zu strukturieren.

1.1 Was ist ein Textsatzsystem wie Typst?

Bekannte Textverarbeitungsprogramme wie Microsoft Word sind sogenannte WYSIWYG-Editoren. Das Akronym bedeutet: *What you see is what you get*. Ein Vorteil dieser Art von Programmen ist, dass das Resultat auf dem Bildschirm weitgehend so aussieht wie nachher im Druck. Allerdings müssen dabei alle Formatierungen manuell angepasst werden – etwa Schriftgröße, Abstände oder Nummerierung von Überschriften.

Einen anderen Weg verfolgen Textsatzsysteme wie z.B. L^AT_EX oder Typst. Hier werden einmalig Richtlinien für das Layout in einer Vorlage (*template*) definiert. Anschliessend wird der Textinhalt mit Markierungen dem Textsatzsystem übergeben. Die Markierungen dienen dazu, die Funktion des Texts zu beschreiben (beispielsweise Titel, Untertitel, Abbildung, Zitat, ...). Das Textsatzsystem übernimmt dann automatisch die korrekte Formatierung gemäss den Vorgaben.

1.2 Für welche Arbeiten ist Typst geeignet, für welche eher nicht?

Ein Textsatzsystem bietet viele Vorteile, aber auch gewisse Nachteile.

Das Textsatzsystem ist **nicht** für die Arbeit geeignet, wenn:

- die Abgabe in wenigen Tagen ist und keine Erfahrungen mit Typst vorhanden sind (dann ist das am besten bekannte Textverarbeitungsprogramm die beste Wahl)
- das Layout pixelgenau kontrollierbar sein soll und eine individuelle künstlerische Gestaltung gewünscht wird (dann sind professionelle DTP-Programme wie Adobe InDesign die bessere Wahl)
- eine starke Abneigung gegen Programmieren und allem, was danach aussieht, vorliegt

Das Textsatzsystem ist **sehr gut** für die Arbeit geeignet, wenn:

- die Bereitschaft da ist, ein wenig Zeit für die Einarbeitung zu investieren (dafür fällt der Zeitaufwand für die Formatierung in der Endphase vor der Abgabe weg)
- viele Formeln (Mathematik, Physik, Chemie) in der Arbeit vorkommen
- viele Literatur-Quellen zitiert werden sollen
- der Inhalt im Zentrum stehen soll, nicht die Formatierung

1.3 Inhalt dieses Dokuments

Folgende Punkte werden in diesem Dokument behandelt:

1. Auf welchen Grundkonzepten basiert Typst?
2. Wie funktioniert das Formatieren von Text?
3. Wie können Grafiken und Tabellen eingebunden werden?
4. Welche Funktionen bietet das kzn-ma der Kantonsschule Zürich Nord sonst noch?

2 Das Textsatzsystem Typst

2.1 Textformatierung

Der eigentliche Text wird in Typst als Markdown (vgl. <https://www.markdownguide.org>) geschrieben. Markdown ist unformatierter Text mit Textzeichen, die die Formatierung steuern. Die wichtigsten Formatierungen in Markdown sind:

- **Titel:** hier werden Gleichheitszeichen vor den Titel gestellt. Je mehr, desto tiefer ist die Titelebene:

```
= Haupttitel  
== Untertitel  
=== Untertitel eine Ebene tiefer
```

- **Fettdruck** und *kursiv*:

```
*fettdruck* und _kursiv_
```

- **Listen** werden mit - oder + markiert:

```
- Aufzählung 1  
- Aufzählung 2  
- Aufzählung 3  
  
+ Nummerierte Liste 1  
+ Nummerierte Liste 2  
+ Nummerierte Liste 3
```

2.2 Funktionen für die Formatierung

Komplexere Formatierungsanweisungen werden in Typst mit Hilfe von Funktionen umgesetzt. Solche Funktionen werden zum Teil vom System zur Verfügung gestellt, können aber auch selbst definiert werden, wie z.B. im hier verwendeten kzn-ma.

2.2.1 Funktionsaufruf aus dem Text

Eine Funktion wird im Text so aufgerufen:

```
#funktionsname()
```

Ein Beispiel ist der Seitenumbruch:

```
#pagebreak()
```

Funktionen können auch Argumente entgegennehmen, z.B. die Art des Seitenumbruchs. Der folgende Seitenumbruch fügt eine Leerseite ein, bis eine ungerade Seitenzahl folgt (to: "odd"), dies aber nur, wenn wirklich notwendig, daher (weak: true):

```
#pagebreak(to: "odd", weak: true)
```

2.2.2 Spezielle Funktionen

Vom Typst-System werden Funktionen zur Verfügung gestellt. Drei davon sind unverzichtbar:

#let

Mit der Funktion **#let** werden eigene Variablen definiert:

```
#let meinTitel = "Der Titel"
```

Diese Variable kann dann überall im Text aufgerufen werden mit:

```
#meinTitel
```

Ganz ähnlich werden auch eigene Funktionen definiert:

```
#let meineFunktion() = {
  pagebreak()
}
```

Diese Funktion kann nun überall, wo sie benötigt wird, aufgerufen werden:

```
#meineFunktion()
```

#set

Die Funktion **#set** dient dazu, verschiedene Formatierungsparameter zu ändern, z.B.:

```
#set text(
  font: "New Computer Modern",
  size: 10pt,
)
```

Hier wird die Schriftart und die Schriftgrösse gesetzt.

#show

Die Funktion **#show** ermöglicht es, das Aussehen von Elementen dauerhaft zu ändern. Beispiel:

```
#show link: set text(blue)
```

setzt die Farbe für URL-Links auf blau. Es können nicht nur schon vorhandene Elemente mit **#show** verändert werden, es können auch Textausdrücke formatiert und oder ersetzt werden. Der folgende Aufruf sorgt dafür, dass jedes Auftreten des Textes «Typst» blau eingefärbt wird.

```
#show "Typst": set text(blue)
```

Gültigkeit der Funktionsaufrufe

Wenn die Definitionen, die mit **#let**, **#set** oder **#show** gemacht werden, gelten ab der Stelle, wo die Funktionen aufgerufen werden. Wenn die Anweisungen für das ganze Dokument gelten sollen, müssen die

Aufrufe am Anfang des Dokuments stehen oder in einer externen Datei aufgerufen werden, die am Anfang vom Textdokument mit `#import "Dateipfad.typ":*` eingebunden wird. Dazu mehr im **Kap. 3.1**.

2.3 Verweise und Fussnoten

Verweise auf Kapitel, Abbildungen, Tabellen oder Literaturreferenzen werden in Typst alle gleich erzeugt:

```
@Ziel-Bezeichnung
```

Wenn das Ziel eines Verweises ein Element im Text ist (also Abbildungen, Tabellen, Kapitel etc.) muss am entsprechenden Ort ein Anker gesetzt werden:

```
<Ziel-Bezeichnung>
```

Konkret kann das so aussehen. Zuerst wird der Kapiteltitel als mögliches Verweisziel markiert:

```
= Das erste Kapitel <ersteskapitel>
```

Viele Textzeilen später wird ein Verweis auf diese Kapitelnummer benötigt, der mit @ erzeugt wird:

```
Wie in @ersteskapitel beschrieben...
```

Bei der Wahl der Zielbezeichnung ist man frei, es dürfen allerdings keine Leerzeichen verwendet werden. Es kann nützlich sein, die Art des Linkziels zu verdeutlichen, z.B. so:

```
<sec:ersteskapitel> // für Kapitel  
<fig:abbildung1>    // für Abbildungen  
<tab:eineTabelle>   // für Tabellen
```

Der Vorteil ist, dass sofort erkennbar ist, ob eine Literaturreferenz oder ein interner Verweis gesetzt wird. Literaturreferenzen funktionieren nämlich grundsätzlich gleich, die Bezeichnung für das Ziel befindet sich aber in der separaten Literatur-Datenbank, dazu mehr in **Kap. 3.4**.

2.4 Tabellen

Tabellen werden verwendet, um beispielsweise Ergebnisse übersichtlich darzustellen, oder für eine Gegenüberstellung. Meist besteht sie aus einem Kopf, der die einzelnen Spalten bezeichnet und dem Inhalt. Diese können in Typst elegant gesetzt werden:

```
#figure(  
  // Diese Tabelle wird in eine figure-Umgebung eingebettet,  
  // was das Anbringen einer Legende erlaubt  
  table(  
    columns: 4,  
    // Anzahl Spalten; alternativ: (2cm, 3cm, 3cm, 3cm)  
    align: (left, center, center, center),  
    // Ausrichtung der einzelnen Spalten  
    fill: (_, y) => if y == 0 { blue.transparentize(80%) },  
    // Hintergrundfarbe: nur die erste Zeile (y=0) wird eingefärbt
```



```

stroke: (_, y) => (
  x: none,
  top: if y == 0 { 1pt } else { if y == 1 { 0.5pt } else { 0pt } },
  bottom: 1pt,
),
// Tabellenlinien werden einzeln gestaltet
table.header([], [Länge in m], [Breite in m], [Höhe in m]),
// Titelzeile wird bei Seitenumbruch wiederholt
[Tisch],    [1.20], [1.20], [0.80],
[Schrank],  [1.15], [0.65], [2.20],
[Bett],     [2.10], [0.90], [0.25],
),
caption: [Abmessungen einzelner Möbelstücke],
kind: table,
)<tabl>

```

Die gesetzte Tabelle:

	Länge in m	Breite in m	Höhe in m
Tisch	1.20	1.20	0.80
Schrank	1.15	0.65	2.20
Bett	2.10	0.90	0.25

Tab. 1: Abmessungen einzelner Möbelstücke

2.5 Illustrationen / Bilder

Bilder (Bitmap-Formate und Vektorbilder im SVG-Format) werden über den `#image()`-Befehl eingefügt. Dies erfolgt meist wieder innerhalb von `#figure()`, um auch bei Bildern eine aussagekräftige Legende anzubringen:

```

#figure(
  image("img/image.jpeg", width: 80%),
  caption: [Kantonsschule Zürich Nord],
  kind: figure,
)<fig1>

```

Als Argument kann die gewünschte Breite des Bildes (im Beispiel: `width: 80%`) angegeben werden, wobei relative Längen in Prozent der verfügbaren Breite oder absolute (beispielsweise in cm oder mm) angegeben werden können.

2.6 Formeln

Bei naturwissenschaftlichen Arbeiten müssen oft Formeln gesetzt werden. Diese können sowohl innerhalb des Lauftextes oder freistehend auftreten. Um eine Formel im Textfluss zu setzen, wird sie von einem Dollarzeichen eingeschlossen: `$a=2g$`. Dies wird meist für eine Wertangabe verwendet. Im Lauftext sollte auf den Einsatz von Brüchen verzichtet werden. Eigentliche Umrechnungen oder Formeln werden freistehend gesetzt. Dies

wird erreicht, indem ein Leerschlag nach und vor dem Dollarzeichen getippt wird: `$ arrow(F)_"res"=m
arrow(a) $`.

$$\vec{F}_{\text{res}} = m\vec{a}$$

Da der Formelsatz äusserst mächtig ist, wird hier auf eine Auflistung der Befehle verzichtet und auf die offizielle Dokumentation verwiesen.

2.7 Einheiten

Das Zusatzpaket `unify` erlaubt es, Einheiten typografisch korrekt zu setzen. Die Einheiten können sowohl innerhalb einer Formel als auch im Lauftext angebracht werden, wie folgendes Beispiel zeigt:

```
#import "@preview/unify:0.7.1": unit, qty, num
```

Ein Fahrzeug der Masse `#qty("1230", "kg")` wird mit einer resultierenden Kraft von `#qty("15", "kilo Newton")` gezogen. Wie gross ist die Beschleunigung, die das Fahrzeug dadurch erfährt? Lösung in `#unit("meter per second squared")` angeben.

```
$ a = F_"res"/m = qty("15.0", "kN")/qty("1230", "kilo gram")  
    = qty("15.0e3", "N")/qty("1230", "kilo gram")  
    = qty("12.2", "m/s^2") $
```

Wie im Beispiel ersichtlich ist, können die Einheiten entweder mit dem jeweiligen Symbol oder dem ausgeschriebenen Namen notiert werden, jedoch keine Mischformen davon: `#unit("m per second squared")` klappt beispielsweise nicht. Innerhalb des Formelsatzes ist das Notieren des Gartenhages vor Befehlen fakultativ und wurde im Beispiel weggelassen. Mit `#num()` kann nur eine Masszahl und mit `#unit()` nur die Einheit gesetzt werden.

3 Das kzn-ma.typ konkret

Das kzn-ma versucht, wo immer möglich, Standardlösungen zu verwenden. Es werden aber einige Einstellungen vorgenommen, um das Layout an die Anforderungen einer Abschlussarbeit der Kantonsschule Zürich Nord (KZN) anzupassen. Weiter werden Hilfsfunktionen zur Verfügung gestellt, die die Layout-Gestaltung erleichtern sollen. Im folgenden Kapitel werden die Handhabung des Templates und die wichtigsten Funktionen kurz mit Beispielen vorgestellt.

3.1 Ein neues Projekt anlegen

Am einfachsten wird das Projekt gestartet, indem auf der Typst-Startseite «Start from template» gewählt wird. Ein Projekt besteht dann aus folgenden Ordnern und Dokumenten:

```
project/
├─ img/                // Ordner mit Bilddateien
├─ main.typ            // Hauptdokument mit allen Einstellungen
├─ main-matter-de.typ  // Hauptteil der Arbeit
├─ appendix-de.typ     // Anhang (oder Anhänge)
└─ mendeley.bib        // Literaturdatenbank
```

Alle Einstellungen werden im File `main.typ` vorgenommen. Nach dem Anlegen des Projekts sind bereits Standardwerte eingetragen.

3.2 Anpassen von Textvariablen

Im Dokument `main.typ` müssen einige Variablen an die eigene Arbeit angepasst werden, z.B. Titel, Autoren und Betreuende.

Andere Variablen werden vom Template in mehrsprachigen Varianten zur Verfügung gestellt. Diese werden mit

```
... = localize(Variablenname)
```

abgerufen.

Alle Variablen können auch bei Bedarf durch eigene Ausdrücke überschrieben werden, z.B.:

```
#let toc-title = "Inhaltsübersicht"
```

Gewisse Variablen können auch deaktiviert werden:

```
#let supervisors = none
```

```
8 // Namen der Autorinnen und Autoren
9 #let school = "Kantonsschule Zürich Nord"
10 #let title = localize(title)
11 #let subtitle = localize(subtitle)
12 #let authors = ("Christian Prim", "Lukas Zuberbühler")
13 #let supervisors = none
```

```
14 #let date = datetime(day: 23, month: 2, year: 2026).display("[day].[month].  
[year]")  
15 #let thesis-type = localize(thesis-type-beginners-guide)  
16 #let written-by = localize(written-by)  
17 #let supervised-by = none  
18 #let submitted-on = "Version"  
19  
20 // Quellenangaben  
21 #let biblio-style = "ieee"  
22 #let biblio-file = "mendeley.bib"  
23  
24 // Inhaltsdateien  
25 #let content-file = "main-matter.typ"  
26 #let appendix-file = "appendix.typ"  
27  
28 // Bezeichnungen für Abschnitte und Verzeichnisse  
29 #let abstract-title = localize(abstract-title)  
30 #let preface-title = localize(preface-title)  
31 #let ai-declaration-title = localize(ai-declaration-title)  
32 #let acknowledgments-title = localize(acknowledgments-title)  
33 #let toc-title = localize(toc-title)  
34 #let lof-title = localize(lof-title)  
35 #let lot-title = localize(lot-title)  
36 #let biblio-title = localize(biblio-title)  
37 #let appendix-title = localize(appendix-title)  
38  
39 // Kurzbezeichnungen  
40 #let heading-desc = localize(heading-desc)  
41 #let fig-desc = localize(fig-desc)  
42 #let tab-desc = localize(tab-desc)  
43 #let appendix-desc = localize(appendix-desc)
```

3.3 Anpassen des Layouts

Für die Layoutanpassungen werden mehrere *Dictionaries* verwendet. Ein *Dictionary* ist ein Datentyp in Typst und erlaubt es, mehrere Variablen und/oder Funktionen «zu verpacken».

3.3.1 Grundsätzliche Layoutanpassungen

```

56 #let layout-def = (
57   paper: "a4",
58   font-size: 11pt,
59   margin: (top: 2cm, inside: 3cm, outside: 2cm, bottom: 2.5cm),
60   numbering: "1",
61   mainFont: "EB Garamond",
62   monoFont: "New Computer Modern Mono",
63   mathFont: "Libertinus Math",
64   copystop: false,
65   header: kzn-header(even-text: [#title]),
66   footer: kzn-footer(footer-text: [#school]),
67   link-color: green,
68   heading-desc: heading-desc,
69   fig-desc: fig-desc,
70   tab-desc: tab-desc,
71   language: "de",
72   language-region: "CH",
73 )

```

Die Einträge hier müssen nicht unbedingt geändert werden, können aber durch eigene Inhalte überschrieben werden. Einige Einträge haben etwas speziellere Eigenschaften:

- Zeile 85: Hier wird die Sprache gesetzt. Unterstützt werden: de, en, fr, it, es
- Zeile 86: Hier wird die Region gesetzt, was sich z.B. auf die Anführungszeichen auswirkt. Mit "CH" werden Schweizer Anführungszeichen («...») verwendet, mit "DE" deutsche („...“). Zur Verfügung stehen alle ISO 3166-1 alpha-2 Region-Codes [3]
- Zeile 71: Hier kann eine anonymisierte Version der Arbeit ohne Bilder, ohne Titelblatt und ohne persönlich markierte Informationen erstellt werden, indem `copystop = true` gesetzt wird (vgl. **Kap. 3.6**)
- Zeile 74 und 77: Hier werden Funktionen zur Verfügung gestellt, die Kopf- und Fusszeilen korrekt für ein zweiseitiges Layout erstellen. Standard: Titel der Arbeit auf geraden Seiten aussen, aktueller Kapiteltitel auf ungeraden Seiten aussen, Seitenzahl und Schulname in der Fusszeile

Im nächsten Block werden die Verzeichnisse eingerichtet:

- Inhaltsverzeichnis (*table of contents*, **toc**)
- Abbildungsverzeichnis (*list of figures*, **lof**)
- Tabellenverzeichnis (*list of tables*, **lot**)

Standardmässig sind alle Verzeichnisse aktiviert und werden vor der Arbeit gesetzt. Im folgenden Block:

```

#let outline-def = (
  toc: [= #toc-title],

```

```

lof: [= #lof-title],
lot: [= #lot-title],
// numbering: "i",
// toc-depth: 3,
// toc-position: "before",
// toc-outlined: false,
// lof-position: "before",
// lof-outlined: false,
// lot-position: "before",
// lot-outlined: true,
)

```

kann dies geändert werden:

- Ein Verzeichnis wird deaktiviert mit: `lot: none`
- Ein aktives Verzeichnis wird ans Ende der Arbeit gestellt mit: `lot-position: "after"`
- Ein aktives Verzeichnis wird aus dem Inhaltsverzeichnis ausgeschlossen mit: `lot-outlined: false`
- Die Seitennummerierung kann geändert werden: `numbering: "1"` — wenn das Verzeichnis ans Ende der Arbeit gestellt wird, hat diese Definition keine Wirkung
- Standardmässig werden drei Titlebenen ins Inhaltsverzeichnis aufgenommen; das lässt sich z.B. mit `toc-depth: 2` ändern. Alle Titlebenen tiefer als die angegebene Zahl werden aus dem Inhaltsverzeichnis ausgeschlossen und **verlieren auch im Text die Nummerierung**

3.3.2 Titelseite

Das Template stellt zwei vordefinierte Titelseiten-Funktionen zur Verfügung. Welche verwendet wird, wird in der Variable `titlepage-def` festgelegt:

```

#let titlepage-def = (
  content: kznTitlePage() // KZN-Gestaltung mit Hintergrundbild
  // content: simpleTitlePage() // Einfache, textbasierte Variante
)

```

Einfache Titelseite: `simpleTitlePage()`

Die Funktion `simpleTitlePage()` erzeugt eine einfache Titelseite mit den wichtigsten Angaben sowie einem optionalen Bild in der Mitte. Sie verwendet die allgemein definierten Variablen `title`, `subtitle`, `authors`, `date` usw. direkt. Der Code der einfachen Titelseite ist im Dokument `main.typ` sichtbar und kann als Ausgangspunkt für eigene Titelseiten verwendet werden.

KZN-Titelseite: `kznTitlePage()`

Die Funktion `kznTitlePage()` erzeugt eine Titelseite, die dem kantonalen CI-Design nachempfunden ist. Sie wird über `kzn-titlepage()` aus dem Template aufgerufen und kann mit folgenden Parametern angepasst werden:

```

#let kznTitlePage() = kzn-titlepage(
  authors: authors,
  supervisors: supervisors,
)

```

```

title: localize(title),
title-size: 28pt,
subtitle: subtitle,
subtitle-size: 18pt,
date: date,
nord-image: image("img/image.jpeg", height: 100%), // Hintergrundbild
(Dateipfad)
nord-image-source: [ // Quellenangabe zum Bild (optional)
  #localize(cover-image) #link("https://...")
],
nord-color: black, // Farbe der grafischen Elemente
background-color: white, // Hintergrundfarbe der Seite
zh-blue: rgb("009EE0"), // Akzentfarbe (Zürcher Blau)
heading-font: "Lato", // Schriftart für die Titelseite
strings: (
  submitted-on: submitted-on,
  thesis-type: thesis-type,
  written-by: written-by,
  supervised-by: supervised-by,
),
)

```

Soll kein Hintergrundbild verwendet werden, wird der Parameter `nord-image` weggelassen oder auf `none` gesetzt. Die grafischen Elemente erscheinen dann in der gewählten `nord-color`.

Eigene Titelseite

Eine vollständig eigene Titelseite kann als Typst-Funktion definiert und direkt als Inhalt von `titlepage-def` übergeben werden:

```

#let meineTitelseite() = {
  place(top + left, [
    #block(text(weight: "bold", size: 20pt, [Mein Schulname]))
  ])
  v(5cm)
  align(center, [
    #block(text(weight: "bold", size: 28pt, title))
    #block(text(size: 16pt, subtitle))
  ])
}

#let titlepage-def = (
  content: meineTitelseite()
)

```

3.3.3 Definition der Blöcke vor der eigentlichen Arbeit

Vor dem Inhaltsverzeichnis können mehrere Blöcke eingefügt werden, z.B. Abstract, Vorwort, Danksagung oder eine Erklärung zur KI-Nutzung. Diese werden in `frontmatter-def` zusammengestellt:

```
#let frontmatter-def = (  
  content: (  
    myAbstract(),  
    myAIDeclaration(),  
    myAcknowledgments(),  
    myPreface(),  
    myCustomBlock(),  
  ),  
  // numbering: "i", // Römische Seitennummerierung (Standard)  
)
```

Die Reihenfolge der Blöcke in `content: (...)` bestimmt die Reihenfolge im Dokument. Nicht benötigte Blöcke werden einfach aus der Liste entfernt. Jeder Block ist eine Funktion, die Typst-Inhalt zurückgibt:

```
#let myAbstract() = {  
  [= #abstract-title  
  
  Hier steht die kurze Zusammenfassung der Arbeit.  
]  
}
```

Eigene Blöcke können nach demselben Muster definiert und in `content: (...)` aufgenommen werden:

```
#let meinEigenerBlock() = {  
  [= Eigenständigkeitserklärung  
  
  Ich erkläre, dass ich diese Arbeit selbstständig verfasst habe.  
]  
}  
  
#let frontmatter-def = (  
  content: (myAbstract(), meinEigenerBlock(), myAIDeclaration()),  
)
```

Die Seitennummerierung im Frontmatter ist standardmässig mit römischen Kleinbuchstaben (i, ii, iii, ...) eingestellt. Dies kann geändert werden:

```
#let frontmatter-def = (  
  content: (...),  
  numbering: "1", // Arabische Ziffern statt römischer  
)
```

Vordefinierte Blöcke

Das Template stellt folgende Blöcke zur Verfügung, die direkt verwendet werden können:

- `myAbstract()`: Kurze Zusammenfassung der Arbeit
- `myAIDeclaration()`: Erklärung zur Nutzung von KI-Tools (mehrsprachig)
- `myAcknowledgments()`: Danksagung

- myPreface(): Vorwort

3.3.4 Anhänge

Der Anhang wird in der Datei `appendix.typ` verfasst und am Ende von `main.typ` eingebunden. Die Struktur in `main.typ` sieht so aus:

```
#set heading(
  numbering: "A",           // Alphabetische Nummerierung der Anhang-Titel
  outlined: true,          // Anhänge erscheinen im Inhaltsverzeichnis
  supplement: appendix-desc,
)
#counter(heading).update(0) // Neustart der Nummerierung bei A
#pagebreak(weak: true, to: "odd") // Anhang beginnt auf einer rechten Seite

#include appendix-file
```

Der Aufruf `#counter(heading).update(0)` sorgt dafür, dass die Nummerierung neu bei A beginnt, unabhängig von der Kapitelanzahl im Hauptteil. Soll der Anhang keine Nummerierung haben, wird `numbering: none` gesetzt.

Sollen mehrere Anhänge als separate Dateien eingebunden werden, können mehrere `#include`-Aufrufe verwendet werden:

```
#include "appendix-a.typ"
#include "appendix-b.typ"
```

3.4 Literaturreferenzen

Literaturreferenzen werden in einer separaten Datei im BibTeX-Format verwaltet. Der Dateiname wird in `main.typ` festgelegt:

```
#let biblio-file = "mendeley.bib"
```

Die BibTeX-Datei

Eine BibTeX-Datei enthält Einträge für jede Quelle. Ein typischer Eintrag sieht so aus:

```
@article{einstein1905,  
  author = {Einstein, Albert},  
  title  = {Zur Elektrodynamik bewegter Körper},  
  journal = {Annalen der Physik},  
  year   = {1905},  
  volume = {17},  
  pages  = {891--921},  
}  
  
@book{feynman1964,  
  author   = {Feynman, Richard P.},  
  title    = {The Feynman Lectures on Physics},  
  publisher = {Addison-Wesley},  
  year     = {1964},  
}  
  
@misc{typst2026,  
  author = {{Typst GmbH}},  
  title  = {Typst Documentation},  
  year   = {2026},  
  url    = {https://typst.app/docs},  
}
```

Die Bezeichnung in den geschweiften Klammern (z.B. `einstein1905`) ist der Zitierschlüssel, mit dem die Quelle im Text referenziert wird.

Zitieren im Text

Eine Quelle wird im Text mit `@` und dem Zitierschlüssel referenziert:

Die Lichtgeschwindigkeit ist konstant `@einstein1905`.

Wie in `@feynman1964[S.~42]` beschrieben...

Typst formatiert die Referenz automatisch gemäss dem gewählten Zitierstil.

Zitierstile

Der Zitierstil wird in `main.typ` festgelegt:

```
#let biblio-style = "ieee"
```

Zahlreiche Stile sind verfügbar (vgl. <https://typst.app/docs/reference/model/bibliography/>).

Die gebräuchlichsten ohne Fussnoten:

- "ieee"
- "apa"

Mit Fussnoten unten an der Seite:

- "chicago-notes"
- "chicago-fullnotes"
- "chicago-shortened-notes"
- "modern-humanities-research-association-notes"
- "modern-humanities-research-association"
- "turabian-fullnote-8"

Die Wahl des Stils richtet sich nach den Vorgaben der Schule oder des Fachbereichs.

Export aus Literaturverwaltungsprogrammen

Die BibTeX-Datei kann direkt aus Literaturverwaltungsprogrammen exportiert werden:

- **Zotero**: Datei → Bibliothek exportieren → Format: BibTeX
- **Mendeley**: Einträge Auswählen → Export → BibTeX
- **JabRef**: Arbeitet nativ im BibTeX-Format

3.5 Mehrsprachigkeit und `localize()`

Das Template unterstützt die Sprachen Deutsch ("de"), Englisch ("en"), Französisch ("fr"), Italienisch ("it") und Spanisch ("es"). Die aktive Sprache wird in `layout-def` festgelegt:

```
#let layout-def = (  
  ...  
  language: "de",  
  language-region: "CH",  
)
```

Die Funktion `localize()`

Alle im Template vordefinierten Bezeichnungen (Inhaltsverzeichnis, Abstract, Anhang usw.) sind als mehrsprachige Dictionaries definiert und werden mit `localize()` in die aktive Sprache übersetzt:

```
#let toc-title = localize(toc-title)
```

Eigene mehrsprachige Texte können nach demselben Muster als Dictionary definiert werden:

```
#let meinText = (  
  de: [Mein Text auf Deutsch],  
  en: [My text in English],  
  fr: [Mon texte en français],  
)  
  
// Im Dokument:  
#localize(meinText)
```

`localize()` wählt automatisch die Sprache, die in `layout-def` gesetzt ist. Ist die aktive Sprache im Dictionary nicht vorhanden, wird zuerst Deutsch, dann Englisch als Fallback verwendet.

Sprachregion

Der Parameter `language-region` beeinflusst sprachregionale Konventionen, insbesondere Anführungszeichen. Mit `language-region: "CH"` werden Schweizer Anführungszeichen («...») verwendet, mit "DE" deutsche („...“).

3.6 Anonymisierung mit copystop

Für die Einreichung bei Plagiatsprüfungsdiensten kann eine anonymisierte Version des Dokuments erstellt werden, bei der persönliche Angaben und Bilder ausgeblendet werden. Dies wird mit dem Parameter `copystop` in `layout-def` aktiviert:

```
#let layout-def = (
  ...
  copystop: true,    // Anonymisierter Modus aktiv
  // copystop: false, // Normaler Modus
)
```

Im anonymisierten Modus werden folgende Elemente ausgeblendet bzw. ersetzt:

- Die Titelseite wird durch eine vereinfachte Version mit Titel und dem Hinweis «*Anonyme Version*» ersetzt
- Alle Bilder werden durch ein Kreuz-Symbol ersetzt
- Alle mit `#private()` markierten Inhalte werden ausgeblendet

Die Funktion `#private()`

Inhalte, die im anonymisierten Modus nicht erscheinen sollen, werden mit `#private()` markiert:

```
// Persönlicher Name im Fliesstext
Wie #private([Maria Muster]) in ihrer Analyse zeigt...

// Link zur eigenen Website
#private(link("https://meine-seite.ch"))

// Ganzer Absatz
#private([
  Dieser Abschnitt enthält persönliche Informationen,
  die bei der Plagiatsprüfung nicht sichtbar sein sollen.
])
```

Im normalen Modus (`copystop: false`) wird der Inhalt von `#private()` vollständig angezeigt. Im anonymisierten Modus (`copystop: true`) wird er durch nichts ersetzt – der umgebende Text fließt nahtlos zusammen.

Empfehlung: Namen, Danksagungsadressen und persönliche URLs konsequent mit `#private()` markieren, damit die anonymisierte Version mit einem einzigen Parameter-Wechsel erstellt werden kann.

3.7 Erweiterungen der #figure-Umgebung

Das KZN-Template benutzt die vom Typst-Standard vorgesehene Syntax zum Einbinden von Abbildungen. Als Erweiterung können Abbildungen mit mehreren Teilabbildungen erstellt werden, indem ein `#grid()` angelegt wird. Es sind beliebig viele Spalten und Zeilen definierbar, allerdings ist kein Seitenumbruch möglich, daher sind mehr als zwei Spalten/Zeilen kaum sinnvoll. Ein weiteres Beispiel befindet sich in **Anh. D**.

```
#figure(
  align(center)[
    #grid(
      columns: (0.5fr, 0.5fr),
      gutter: 7mm,
      align: bottom,
      [
        #figure(
          align(center)[#image("Abbildung_a.pdf", width: 100%)],
          caption: [Abbildung (a)],
          kind: "subfigure",
        )<figla>
      ],
      [
        #figure(
          align(center)[#image("Abbildung_b.pdf", width: 100%)],
          caption: [Abbildung (b)],
          kind: "subfigure",
        )<figlb>
      ],
      [
        #figure(
          align(center)[#image("Abbildung_c.pdf", width: 100%)],
          caption: [Abbildung (c)],
          kind: "subfigure",
        )<figlc>
      ],
      [
        #figure(
          align(center)[#image("Abbildung_d.pdf", width: 100%)],
          caption: [Abbildung (d)],
          kind: "subfigure",
        )<figld>
      ],
    ],
  ),
  caption: [Abbildung mit mehreren Teilabbildungen.],
  kind: figure,
)<fig:abbildung>
```

A

(a) Der Buchstabe A

B

(b) Der Buchstabe B

C

(c) Der Buchstabe C

D

(d) Der Buchstabe D

Abb. 1: Abbildung mit mehreren Teilabbildungen.

Die einzelnen Teilabbildungen können referenziert werden; für die vollständige Referenz muss auch die einbettende `#figure` referenziert werden:

```
@fig:abbildung @figla
```

ergibt:

Abb. 1 (a)

4 Diskussion und Ausblick

Das vorliegende Template deckt die typischen Anforderungen einer Maturitätsarbeit ab: strukturierter Textsatz, Formeln, Abbildungen, Tabellen, Literaturverwaltung und mehrsprachige Unterstützung. Einige Bereiche bieten jedoch Potenzial für Weiterentwicklungen, und für den Einstieg in Typst stehen zahlreiche Ressourcen zur Verfügung.

4.1 Weiterentwicklung des Templates

Das Template wird laufend weiterentwickelt. Geplante Erweiterungen umfassen zusätzliche vordefinierte Titelseiten-Varianten, eine vereinfachte Einbindung von Code-Listings mit Syntaxhervorhebung sowie verbesserte Unterstützung für mathematische Beweise und Definitionen als formatierte Blöcke. Rückmeldungen und Verbesserungsvorschläge können direkt an die Autoren gerichtet werden.

4.2 Ressourcen für den Einstieg

4.2.1 Offizielle Dokumentation

Die wichtigste Anlaufstelle ist die offizielle Typst-Dokumentation unter <https://typst.app/docs>. Sie ist vollständig und enthält für jede Funktion interaktive Beispiele, die direkt im Browser ausprobiert werden können. Besonders nützlich sind:

- **Referenz:** <https://typst.app/docs/reference> – vollständige Auflistung aller Funktionen, Typen und Operatoren
- **Guides:** <https://typst.app/docs/guides> – thematische Anleitungen, u.a. zu Formeln, Seitenlayout und Schriften
- **Tutorial:** <https://typst.app/docs/tutorial> – schrittweiser Einstieg für Neueinsteiger

4.2.2 Pakete und Erweiterungen

Das offizielle Paketverzeichnis unter <https://typst.app/universe> listet alle verfügbaren Zusatzpakete. Für naturwissenschaftliche Arbeiten besonders relevant sind:

- **unify** (<https://typst.app/universe/package/unify>): Typografisch korrekter Einheitensatz (im Template bereits eingebunden)
- **codly** (<https://typst.app/universe/package/codly>): Syntaxhervorhebung für Quellcode (im Template bereits eingebunden)
- **cetz** (<https://typst.app/universe/package/cetz>): Zeichnungen und Diagramme direkt in Typst, ähnlich wie TikZ in L^AT_EX
- **fletcher** (<https://typst.app/universe/package/fletcher>): Flussdiagramme und Graphen
- **plotst** (<https://typst.app/universe/package/plotst>): Einfache Diagramme und Plots

4.2.3 Community und Hilfe

Bei Fragen hilft die aktive Typst-Community weiter:

- **Typst Forum:** <https://forum.typst.app> – offizielle Diskussionsplattform, gut durchsuchbar
- **Discord:** Einladungslink über <https://typst.app/community> – für schnelle Fragen geeignet
- **GitHub:** <https://github.com/typst/typst> – Quellcode, bekannte Fehler und Entwicklungsstand

4.2.4 Vergleich mit L^AT_EX

Für Nutzerinnen und Nutzer mit L^AT_EX - Vorkenntnissen gibt es eine Gegenüberstellung der wichtigsten Syntaxunterschiede unter <https://typst.app/docs/guides/guide-for-latex-users>. Typst ist in vielen Belangen schlanker als L^AT_EX, unterstützt aber noch nicht alle Pakete und Eigenheiten, die in der wissenschaftlichen Community etabliert sind – insbesondere im Bereich chemischer Strukturformeln (ChemDraw-Äquivalente) und komplexer bibliografischer Stile.

4.2.5 Tutorials und Lernmaterial

Neben der offiziellen Dokumentation gibt es eine wachsende Zahl an Tutorials:

- **Typst in 3 Minutes:** <https://github.com/typst/typst#learn> – kompakter Schnelleinstieg im README
- **YouTube-Tutorials:** Eine Suche nach «Typst tutorial» liefert aktuelle Einführungsvideos, die den gesamten Workflow von der Installation bis zum fertigen Dokument zeigen
- **Vorlagen auf Typst Universe:** Unter <https://typst.app/universe/search?kind=template> finden sich zahlreiche fertige Templates als Ausgangspunkt oder Inspirationsquelle

Literatur

- [1] Kantonsschule Zürich Nord, «Kantonsschule Zürich Nord». [Online]. Verfügbar unter: <https://www.kzn.ch/>
- [2] Typst GmbH, «Typst: A markup-based typesetting system and web application». [Online]. Verfügbar unter: <https://typst.app/>
- [3] «ISO 3166-1 alpha-2 - Wikipedia». [Online]. Verfügbar unter: https://en.wikipedia.org/wiki/ISO_3166-1_alpha-2

A Formelsatz

Der Formelsatz in Typst ist eng an L^AT_EX angelehnt, aber mit einfacherer Syntax. Freistehende Formeln werden mit `$ ... $` (mit Leerzeichen vor und nach den Dollarzeichen) gesetzt, Inline-Formeln ohne Leerzeichen.

AA Grundlegende Mathematik

```
// Inline: Pythagoras
Der Satz des Pythagoras lautet  $a^2 + b^2 = c^2$ .

// Freistehend: Quadratische Formel
 $x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ 

// Griechische Buchstaben und Operatoren
 $\alpha + \beta = \gamma, \quad \Delta \omega = 2\pi f$ 

// Summen und Produkte
 $\sum_{k=1}^n k = \frac{n(n+1)}{2}, \quad \prod_{i=1}^n i = n!$ 

// Integrale
 $\int_a^b f(x) dx = F(b) - F(a)$ 

// Partielle Ableitungen
 $\frac{\partial f}{\partial x} = \lim_{h \rightarrow 0} \frac{f(x+h, y) - f(x, y)}{h}$ 

// Vektoren und Spaltenvektor
 $\vec{F} = m \vec{a}, \quad \vec{v} = \begin{pmatrix} v_x \\ v_y \\ v_z \end{pmatrix}$ 

// Matrix
 $\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}$ 
```

Die Ausgabe der freistehenden Formeln:

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

$$\alpha + \beta = \gamma, \quad \Delta\omega = 2\pi f$$

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}, \quad \prod_{i=1}^n i = n!$$

$$\int_a^b f(x) dx = F(b) - F(a)$$

$$\frac{\partial f}{\partial x} = \lim_{h \rightarrow 0} \frac{f(x+h, y) - f(x, y)}{h}$$

$$\vec{F} = m\vec{a}, \quad \vec{v} = \begin{pmatrix} v_x \\ v_y \\ v_z \end{pmatrix}$$

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}$$

AB Physikalische Formeln

```
// Tsiolkowski-Gleichung (Raketengrundgleichung)
$ Delta v = v_g dot.c ln(m_0 / m_1) $

// Maxwell-Gleichungen (differentielle Form)
$ nabla dot.c arrow(E) = rho / epsilon_0, quad
  nabla dot.c arrow(B) = 0 $

$ nabla times arrow(E) = - (partial arrow(B))/(partial t), quad
  nabla times arrow(B) = mu_0 arrow(J) + mu_0 epsilon_0 (partial arrow(E))/(partial
  t) $

// Energiebilanz Looping
$ 1/2 m v_0^2 = 1/2 m v_"top"^2 + m g (2r) $

// Doppler-Effekt
$ f = f_0 / (1 - v/c) $
```

$$\Delta v = v_g \cdot \ln\left(\frac{m_0}{m_1}\right)$$

$$\nabla \cdot \vec{E} = \frac{\rho}{\epsilon_0}, \quad \nabla \cdot \vec{B} = 0$$

$$\nabla \times \vec{E} = -\frac{\partial \vec{B}}{\partial t}, \quad \nabla \times \vec{B} = \mu_0 \vec{J} + \mu_0 \epsilon_0 \frac{\partial \vec{E}}{\partial t}$$

$$f = \frac{f_0}{1 - \frac{v}{c}}$$

AC Einheiten mit unify

```
#import "@preview/unify:0.8.1": unit, qty, num

// Masszahlen mit Einheit
Die Lichtgeschwindigkeit beträgt #qty("2.998e8", "m/s").
Ein Parsec entspricht #qty("3.086e16", "m") oder #qty("3.262", "ly").
```

```
// Einheit ohne Masszahl  
Gib das Ergebnis in #unit("kilo gram meter per second squared") an.  
  
// Einheiten innerhalb von Formeln  
$ a = F/m = qty("15.0", "kN") / qty("1230", "kg")  
      = qty("12.2", "m/s^2") $  
  
// SI-Präfixe: ausgeschrieben und als Symbol sind gleichwertig  
#qty("100", "mega pascal")  
#qty("100", "MPa")
```

B Chemische Formeln und Reaktionsgleichungen

Für den Chemiesatz steht das Paket `typsium` (<https://typst.app/universe/package/typsium>) zur Verfügung. Es wird analog zu `mhchem` in $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ über die Funktion `#ce[...]` bedient und muss zu Beginn des Dokuments importiert werden:

```
#import "@preview/typsium:0.3.2": ce
```

Achtung: Das Paket `typsium` ist teilweise nicht kompatibel mit dem Paket `unify`. Daher nur die Funktionen einbinden, die auch wirklich benötigt werden — hier z.B. nur `ce`.

BA Summenformeln und Ionen

Summenformeln werden direkt in `#ce[...]` eingegeben. Indizes, Ladungen und Oxidationszahlen werden automatisch korrekt gesetzt:

```
// Einfache Summenformeln
#ce[H2O] #h(1em) #ce[CO2] #h(1em) #ce[H2SO4]

// Ionen mit Ladung
#ce[Na+] #h(1em) #ce[Cl-] #h(1em) #ce[SO4^2-] #h(1em) #ce[NH4+] #h(1em) #ce[Fe^3+]

// Aggregatzustände
#ce[NaCl(s)] #h(1em) #ce[HCl(g)] #h(1em) #ce[NaOH(aq)]
```

H_2O CO_2 H_2SO_4

Na^+ Cl^- SO_4^{2-} NH_4^+ Fe^{3+}

NaCl(s) HCl(g) NaOH(aq)

BB Reaktionsgleichungen

Reaktionspfeile werden direkt als ASCII-Sequenzen geschrieben:

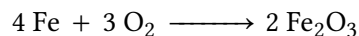
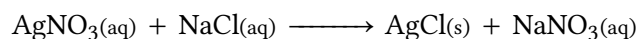
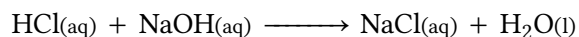
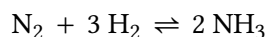
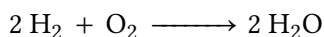
```
// Synthese von Wasser
#ce[2H2 + O2 -> 2H2O]

// Gleichgewichtsreaktion: Haber-Bosch-Verfahren
#ce[N2 + 3H2 <=> 2NH3]

// Säure-Base-Reaktion: Neutralisation
#ce[HCl(aq) + NaOH(aq) -> NaCl(aq) + H2O(l)]

// Fällungsreaktion
#ce[AgNO3(aq) + NaCl(aq) -> AgCl(s) + NaNO3(aq)]

// Redoxreaktion: Korrosion von Eisen
#ce[4Fe + 3O2 -> 2Fe2O3]
```

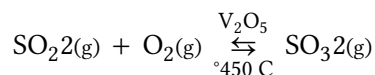


BC Reaktionsgleichung als freistehender Block

Für längere Reaktionsgleichungen oder wenn eine Nummerierung gewünscht wird, kann `#ce` in eine figure-Umgebung eingebettet werden:

```
#figure(
  $ ce("2S02(g) + O2(g) <->[V205][450°C] 2S03(g)") $,
  caption: [Katalytische Oxidation von Schwefeldioxid im Kontaktverfahren.],
  kind: "equation",
  supplement: [Gl.],
)<gl:kontaktverfahren>
```

Wie in `@gl:kontaktverfahren` dargestellt...



Gl. 1: Katalytische Oxidation von Schwefeldioxid im Kontaktverfahren.

Strukturformeln (Lewis-Strukturen, Skelettformeln) sind mit `typsium` nicht darstellbar. Dafür steht das Paket `alchemist` (<https://typst.app/universe/package/alchemist>) zur Verfügung, das auf `cetz` aufbaut und eine ähnliche Syntax wie `chemfig` in $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ bietet.

C Diakritische Zeichen und nicht-lateinische Schriften

Typst verarbeitet Text als Unicode (UTF-8). Alle Sonderzeichen können direkt im Quelltext eingegeben werden – eine besondere Deklaration ist nicht nötig, solange die verwendete Schriftart die entsprechenden Zeichen enthält.

CA Romanische Sprachen

Akzente und Sonderzeichen romanischer Sprachen werden direkt eingegeben:

```
// Französisch
Liberté, égalité, fraternité.
L'œuvre de Molière est représentative du théâtre classique français.
Jean-François a étudié à l'école à Genève.

// Spanisch
La investigación científica requiere método y paciencia.
El niño pequeño aprendió el español con facilidad.
¿Cómo están ustedes? – Muy bien, gracias.

// Portugiesisch
A investigação científica exige rigor metodológico.
São Paulo é a maior cidade do Brasil.
A língua portuguesa tem muita variação regional.

// Italienisch
La ricerca scientifica richiede metodo e pazienza.
Il caffè e la piazza sono elementi centrali della vita italiana.
```

Liberté, égalité, fraternité. L'œuvre de Molière est représentative du théâtre classique français.

La investigación científica requiere método y paciencia. ¿Cómo están ustedes?

A língua portuguesa tem muita variação regional.

La ricerca scientifica richiede metodo e pazienza. Il caffè e la piazza sono elementi centrali della vita italiana.

CB Latein

Lateinische Texte benötigen keine Sonderzeichen, ausser bei mittelalterlichem Latein mit Ligaturen:

```
// Klassisches Latein
Gallia est omnis divisa in partes tres, quarum unam incolunt Belgae,
aliam Aquitani, tertiam qui ipsorum lingua Celtae, nostra Galli appellantur.

// Mittelalterliches Latein mit Ligaturen (direkt als Unicode)
Æneas, filius Veneris, ex Troia fugit.
Cælum, non animum, mutant qui trans mare currunt.

// Makrons für Vokalquantitäten (Aussprache)
```



```
// In EB Garamond und ähnlichen Fonts vorhanden:
```

```
Rōma aeterna. Amō, amās, amat.
```

Gallia est omnis divisa in partes tres, quarum unam incolunt Belgae, aliam Aquitani, tertiam qui ipsorum lingua Celtae, nostra Galli appellantur.

Æneas, filius Veneris, ex Troia fugit. Coelum, non animum, mutant qui trans mare currunt.

Rōma aeterna. Amō, amās, amat.

CC Griechisch

Für griechische Textpassagen (nicht Formeln) muss die Textsprache temporär gesetzt werden, damit Silbentrennung und Typografie korrekt funktionieren:

```
// Kurze Einbettung im deutschen Text
```

```
Der Begriff #text(lang: "el")[φιλοσοφία] (philosophía) bedeutet wörtlich  
«Liebe zur Weisheit».
```

```
// Altgriechisch
```

```
#text(lang: "el") [  
  – Χαῖρε, ὦ φίλε. πῶς ἔχεις;  
  – Εὖ ἔχω, ὦ Σώκρατες. καὶ σύ;  
  – Καὶ ἐγὼ εὖ. τί πράττεις σήμερον;  
]
```

```
// Modernes Griechisch
```

```
#text(lang: "el") [  
  – Γεια σου! Πώς είσαι;  
  – Καλά, ευχαριστώ. Εσύ;  
  – Πολύ καλά. Τι κάνεις σήμερα;  
  – Πηγαίνω στην αγορά. Τα λέμε αργότερα!  
]
```

Der Begriff φιλοσοφία (philosophía) bedeutet wörtlich «Liebe zur Weisheit».

— Χαῖρε, ὦ φίλε. πῶς ἔχεις; — Εὖ ἔχω, ὦ Σώκρατες. καὶ σύ; — Καὶ ἐγὼ εὖ. τί πράττεις σήμερον;

— Γεια σου! Πώς είσαι; — Καλά, ευχαριστώ. Εσύ; — Πολύ καλά. Τι κάνεις σήμερα; — Πηγαίνω στην αγορά. Τα λέμε αργότερα!

CD Arabisch

Arabisch wird von rechts nach links geschrieben (*right-to-left*, RTL). Typst behandelt die Textrichtung automatisch, wenn die Sprache korrekt gesetzt ist:

```
// Eingebettetes Wort im deutschen Text
```

```
Das arabische Wort #text(lang: "ar")[مرحبا] (marḥaban) bedeutet «Hallo».
```

```
// Alltagskommunikation auf Arabisch (vokalisiert)
```

```
// Textrichtung wird automatisch umgeschaltet
#text(lang: "ar") [
  - مَرْحَباً! كَيْفَ خَالَكَ؟
  - يَحْيَى، شُكْرًا. وَأَنْتَ؟
]
```

Das arabische Wort **مرحبا** (marḥaban) bedeutet «Hallo».

— مَرْحَباً! كَيْفَ خَالَكَ؟ — يَحْيَى، شُكْرًا. وَأَنْتَ؟

Hinweis zur Schriftart: Die Standardschrift *EB Garamond* unterstützt Griechisch und Lateinisch mit Ligaturen vollständig. Für Arabisch empfiehlt sich eine dedizierte arabische Schriftart wie *Noto Naskh Arabic*, die separat geladen werden kann:

```
// Schriftart nur für diesen Block wechseln
#text(font: "Noto Naskh Arabic", lang: "ar") [مَرْحَباً! كَيْفَ خَالَكَ؟]
```

مَرْحَباً! كَيْفَ خَالَكَ؟

D Abbildungsbeispiele

DA Einfache Abbildung

Eine einzelne Abbildung wird mit `#figure()` und `kind: figure` eingebunden. Die Legende erscheint automatisch unterhalb der Abbildung und wird im Abbildungsverzeichnis aufgeführt.

```
#figure(
  image("img/image.jpeg", width: 80%),
  caption: [Kantonsschule Zürich Nord, Ansicht von der Birchstrasse.],
  kind: figure,
)<fig:kzn-gebaeude>
```

Wie in `@fig:kzn-gebaeude` zu sehen ist, ...

DB Abbildung als Grid (mehrere Teilabbildungen)

Für mehrere zusammengehörige Abbildungen steht die `kind: "subfigure"`-Erweiterung des Templates zur Verfügung. Die Teilabbildungen werden in einem `#grid()` angeordnet und erhalten eigene Legenden mit Buchstabenkennung (a), (b), ...

```
#figure(
  align(center)[
    #grid(
      columns: (0.5fr, 0.5fr), // Zwei gleich breite Spalten
      gutter: 7mm,             // Abstand zwischen den Teilabbildungen
      align: bottom,
      [
        #figure(
          image("img/bild_a.jpeg", width: 100%),
          caption: [Zustand vor dem Versuch],
          kind: "subfigure",
        )<fig:versuch-a>
      ],
      [
        #figure(
          image("img/bild_b.jpeg", width: 100%),
          caption: [Zustand nach dem Versuch],
          kind: "subfigure",
        )<fig:versuch-b>
      ],
    ],
    caption: [Vergleich des Zustands vor und nach dem Versuch.],
    kind: figure,
  )<fig:versuch>
```

Die Teilabbildungen können einzeln referenziert werden:

In `@fig:versuch` sind beide Zustände gegenübergestellt.
Der Ausgangszustand ist in `@fig:versuch-a`,
das Ergebnis in `@fig:versuch-b` zu sehen.

Für ein 2×2-Raster mit vier Teilabbildungen:

```
#figure(  
  align(center)[  
    #grid(  
      columns: (0.5fr, 0.5fr),  
      gutter: 7mm,  
      align: bottom,  
      [  
        #figure(  
          image("img/a.jpeg", width: 100%),  
          caption: [Variante A],  
          kind: "subfigure",  
        )<fig:var-a>  
      ],  
      [  
        #figure(  
          image("img/b.jpeg", width: 100%),  
          caption: [Variante B],  
          kind: "subfigure",  
        )<fig:var-b>  
      ],  
      [  
        #figure(  
          image("img/c.jpeg", width: 100%),  
          caption: [Variante C],  
          kind: "subfigure",  
        )<fig:var-c>  
      ],  
      [  
        #figure(  
          image("img/d.jpeg", width: 100%),  
          caption: [Variante D],  
          kind: "subfigure",  
        )<fig:var-d>  
      ],  
    )  
  ],  
  caption: [Vier Varianten im Vergleich.],  
  kind: figure,  
)<fig:varianten>
```

E Tabellenbeispiele

EA Einfache Tabelle

```
#figure(
  table(
    columns: 4,
    align: (left, center, right, right),
    fill: (_, y) => if y == 0 { blue.transparentize(80%) },
    stroke: (_, y) => (
      x: none,
      top: if y == 0 { 1pt } else { if y == 1 { 0.5pt } else { 0pt } },
      bottom: 1pt,
    ),
    table.header(
      [*Messung*], [*Bedingung*], [*Wert in m/s*], [*Unsicherheit*],
    ),
    [M1], [Raumtemperatur], [343.2], [±0.5],
    [M2], [0 °C], [331.5], [±0.4],
    [M3], [100 °C], [386.0], [±0.6],
  ),
  caption: [Schallgeschwindigkeit unter verschiedenen Bedingungen.],
  kind: table,
)<tab:schall>
```

Messung	Bedingung	Wert in m/s	Unsicherheit
M1	Raumtemperatur	343.2	±0.5
M2	0 °C	331.5	±0.4
M3	100 °C	386.0	±0.6

Tab. 2: Schallgeschwindigkeit unter verschiedenen Bedingungen.

EB Mehrere Tabellen nebeneinander (Grid)

Analog zu den Abbildungen können auch Tabellen nebeneinander gesetzt werden, indem die kind: "subfigure"-Umgebung verwendet wird. Dies ist nützlich, um kompakte Vergleiche darzustellen.

```
#figure(
  align(center)[
    #grid(
      columns: (0.48fr, 0.48fr),
      gutter: 10mm,
      align: bottom,
      [
        #figure(
          table(
            columns: 3,
```

```

    align: (left, right, right),
    fill: (_, y) => if y == 0 { blue.transparentize(80%) },
    stroke: (_, y) => (
      x: none,
      top: if y == 0 { 1pt } else { if y == 1 { 0.5pt } else { 0pt } },
      bottom: 1pt,
    ),
    table.header([*Planet*], [*Masse (kg)*], [*Radius (km)*]),
    [Merkur], [$3.30 times 10^23$], [2440],
    [Venus], [$4.87 times 10^24$], [6052],
    [Erde], [$5.97 times 10^24$], [6371],
    [Mars], [$6.42 times 10^23$], [3390],
  ),
  caption: [Innere Planeten],
  kind: "subfigure",
)<tab:innere-planeten>
],
[
  #figure(
    table(
      columns: 3,
      align: (left, right, right),
      fill: (_, y) => if y == 0 { blue.transparentize(80%) },
      stroke: (_, y) => (
        x: none,
        top: if y == 0 { 1pt } else { if y == 1 { 0.5pt } else { 0pt } },
        bottom: 1pt,
      ),
      table.header([*Planet*], [*Masse (kg)*], [*Radius (km)*]),
      [Jupiter], [$1.90 times 10^27$], [71492],
      [Saturn], [$5.68 times 10^26$], [60268],
      [Uranus], [$8.68 times 10^25$], [25559],
      [Neptun], [$1.02 times 10^26$], [24764],
    ),
    caption: [Äussere Planeten],
    kind: "subfigure",
  )<tab:aeussere-planeten>
],
)
],
caption: [Physikalische Eigenschaften der Planeten des Sonnensystems.],
kind: table,
)<tab:planeten>

```

Die gesetzte Tabelle:

Planet	Masse (kg)	Radius (km)
Merkur	3.30×10^{23}	2440
Venus	4.87×10^{24}	6052
Erde	5.97×10^{24}	6371
Mars	6.42×10^{23}	3390

Planet	Masse (kg)	Radius (km)
Jupiter	1.90×10^{27}	71492
Saturn	5.68×10^{26}	60268
Uranus	8.68×10^{25}	25559
Neptun	1.02×10^{26}	24764

(a) Innere Planeten

(b) Äussere Planeten

Tab. 3: Physikalische Eigenschaften der Planeten des Sonnensystems.

Die Referenzierung funktioniert analog zu Abbildungen. Mit `@tab:planeten` wird die gesamte Tabelle referenziert, mit `@tab:innere-planeten` und `@tab:aeussere-planeten` die jeweiligen Teiltabellen.